Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики
и радиоэлектроники»

Кафедра программного обеспечения информационных технологий

Специальность «Программная инженерия»

Учебная дисциплина «Метрология, стандартизация и сертификация в информационных технологиях»

ОТЧЕТ

по практической работе №2а
«Метрики Джилба»

Подготовили: Гавлев Н.В.
Ст.гр.410902 Джалилова Н.Р.

Проверил: Болтак С.В.

Минск 2026

Язык для анализа: Rust

Задание:

1 - Подготовить код анализируемой программы для расчета метрики Джилба. В коде для расчета метрики Джилба должны присутствовать все операторы цикла заданного языка, а также операторы ветвления, включая оператор множественного выбора.

2 - Написать программу-парсер

Парсер может быть реализован на любом языке программирования и должен иметь графический интерфейс. Для анализа взять подготовленную программу на заданном языке. Результат работы парсера: вывод на экран абсолютной сложности, относительной сложности и максимального уровня вложенности.

Абсолютная сложность по метрике Джилба - это сумма всех ветвлений (операторы условия, циклы, операторы множественного выбора).

Для расчета абсолютной сложности и максимального уровня вложенности оператор множественного выбора заменяется на вложенные друг в друга операторы условия. Селектор, а также ветка по умолчанию, не является условием и не влияет на размер метрики.

Листинг анализируемого кода:

```
fn calculate_score(values: &[i32]) -> i32 {
    let mut score = 0;
    let mut index = 0usize;
    loop {
        if index >= values.len() {
            break;
        }
        let value = values[index];
        if value > 0 {
            score += value;
        } else if value < 0 {
            score -= value.abs();
        } else {
            score += 1;
        }
        index += 1;
    }
    score
}
fn classify(value: i32) -> &'static str {
    match value {
        0 => "zero",
        1 => "one",
        2 => "two",
        3 => "three",
```

```
        _ => "other",
    }
}
fn process(values: &[i32]) -> i32 {
    let mut total = 0;
    for value in values {
        if value % 2 == 0 {
            total += value;
        }
    }
    let mut counter = 0usize;
    while counter < values.len() {
        total += values[counter];
        counter += 1;
    }
    let mut position = 0usize;
    while let Some(value) = values.get(position) {
        if *value > 10 {
            total += 2;
        }
        position += 1;
    }
    let mut number = 0;
    loop {
        number += 1;

        if number > 3 {
            break;
        }
        for _ in 0..1 {
            match number {
                1 => {
                    match number {
                        1 => {
                            if total > 0 {
                                total += 1;
                            }
                        }
                        _ => {}
                    }
                }
                2 => total += 2,
                3 => total += 3,
                _ => total += 0,
            }
```

```
        }
    }
    total
}
fn main() {
    let values = vec![0, 1, 2, 5, 10, 15];
    let score = calculate_score(&values);
    let result = process(&values);
    println!("Score: {}", score);
    println!("Result: {}", result);
    for value in &values {
        println!("{} -> {}", value, classify(*value));
    }
}
```

Результаты расчета метрик Джилба, посчитанные вручную:
Абсолютная сложность CL = 20
Относительная сложность cl = 20/64 = 0,312.
Максимальный уровень вложенности CLI = 3.

Схема алгоритма программы по ГОСТ19.701-90 представлена на рисунках 1, 2, 3, 4.

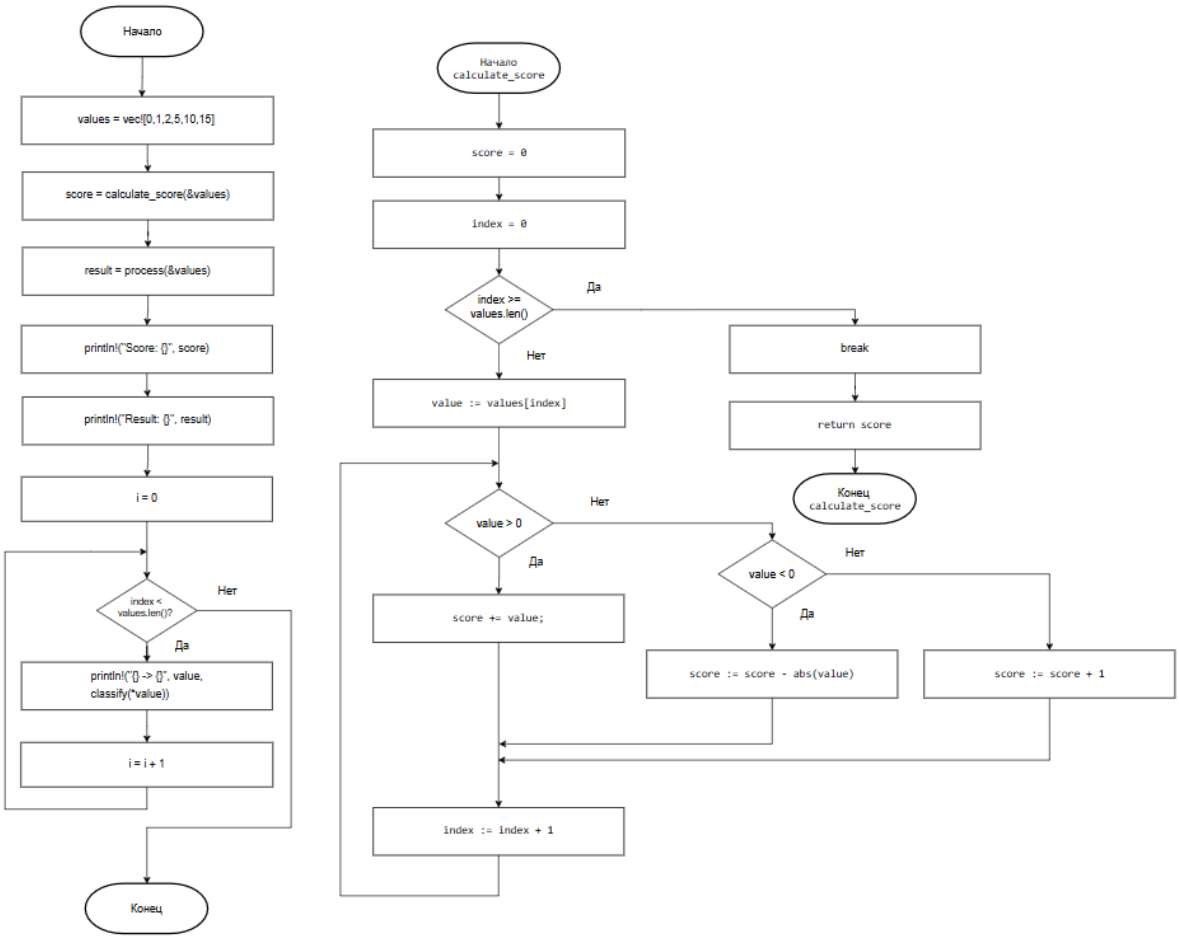


Рисунок 1 – Схема алгоритма программы (функция main и calculate_score)

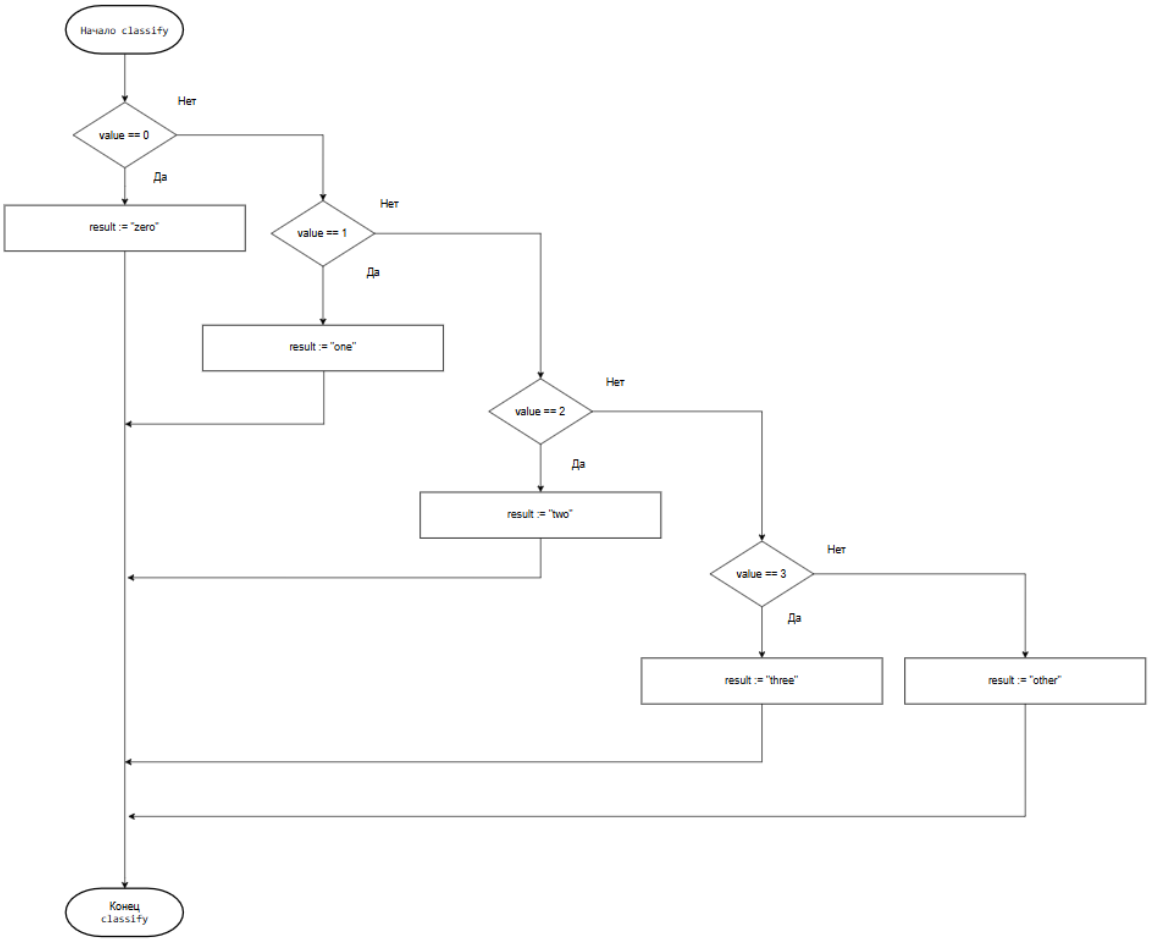


Рисунок 2 – Схема алгоритма программы (функция classify)

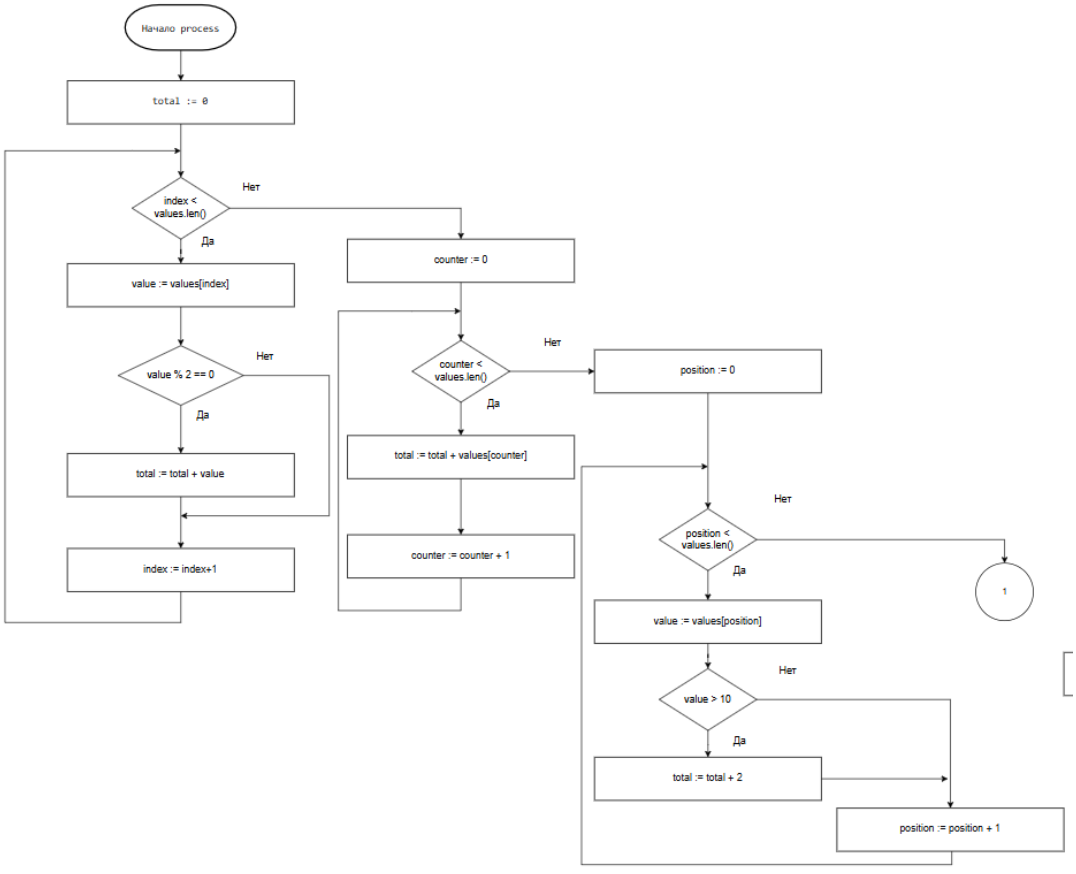


Рисунок 3 – Схема алгоритма программы, часть 1(функция process)

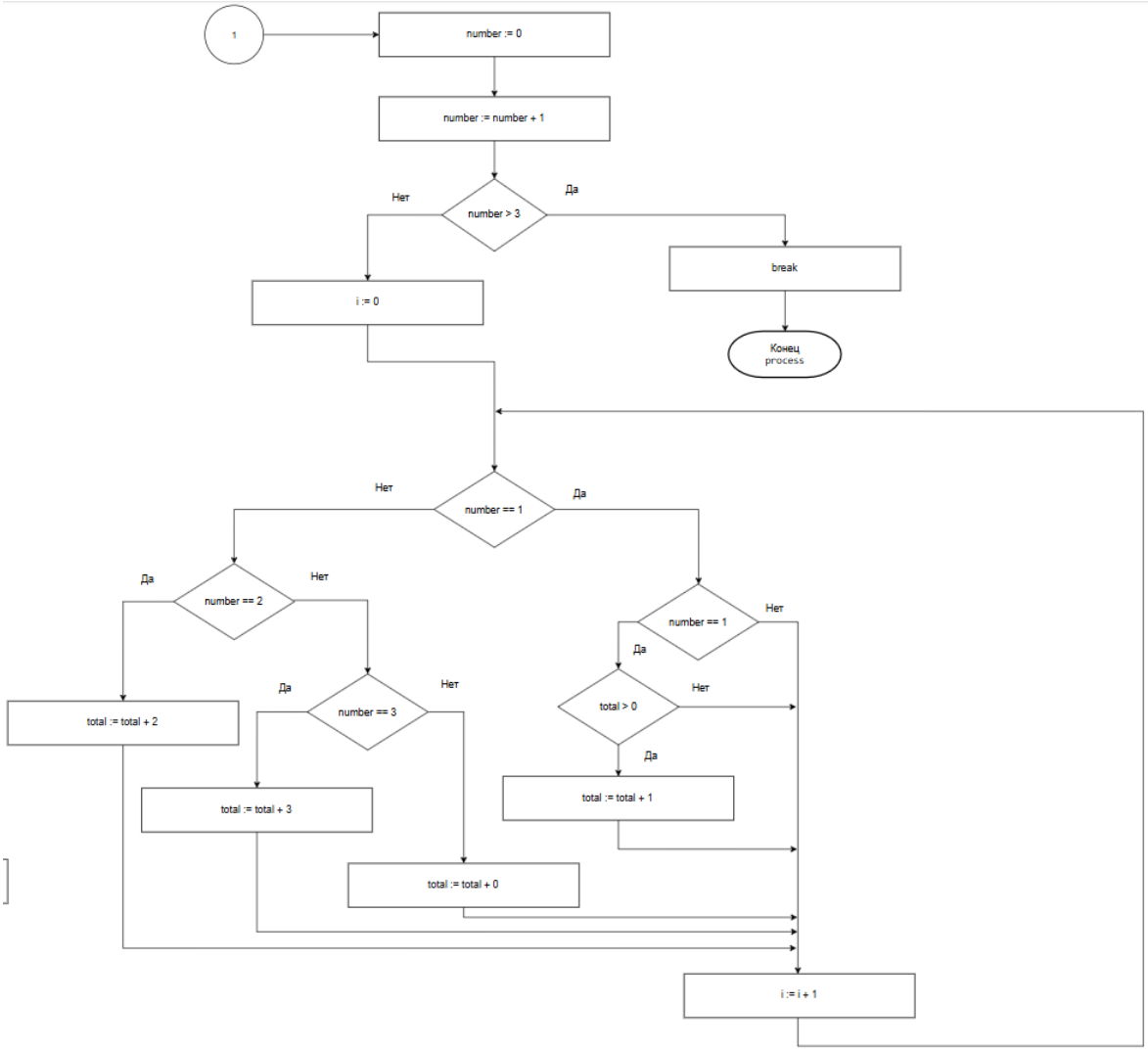


Рисунок 4 – Схема алгоритма программы, часть 2 (функция process)

Скрин с результатами работы парсера (расчет метрик Джилба) представлен на рисунке 5.

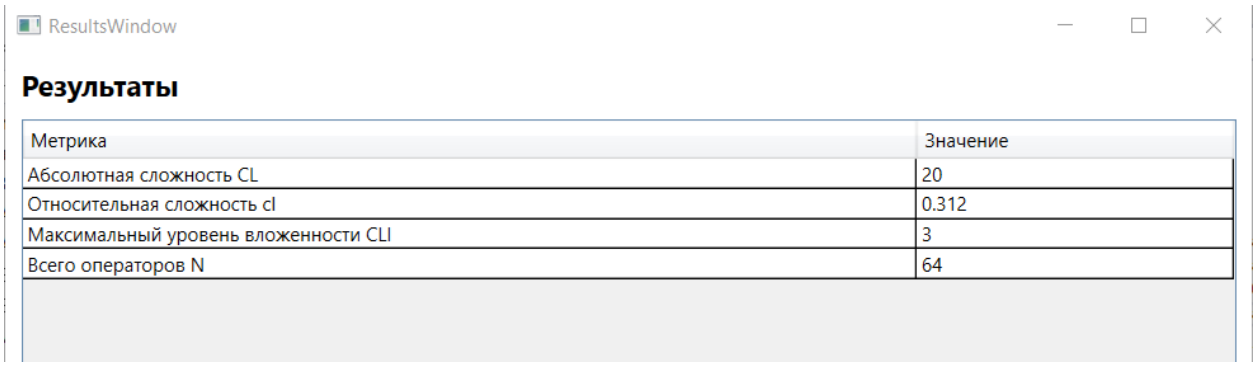


Рисунок 5 – Результаты подсчета метрики Джилба (парсер)